

## Note de Mise à Jour : Optimisation de l'Architecture du Projet

**Objet :** Améliorations de la logique métier et du modèle de données (Application de Facturation).

Bonjour ,

Suite à une analyse approfondie du circuit patient, nous avons identifié des points critiques à intégrer dans notre développement (Backend Django) pour garantir la sécurité des données et la traçabilité financière. Voici les optimisations retenues :

### 1. Mise en place d'un "Verrou de Paiement" (Sécurité Flux)

- **Problématique :** Éviter que des soins ou examens ne soient réalisés sans paiement préalable.
- **Solution :** Les examens ne seront visibles par le **Laborantin** que si le champ `is_paid` de la prescription est à `True`.
- **Logique :** Le Caissier "libère" l'accès technique au dossier pour le laboratoire au moment de la validation de la facture.

### 2. Intégrité des Données : Le "Snapshot" de Prix

- **Problématique :** Si l'administrateur change le prix d'un acte dans le catalogue, les anciennes factures risquent d'être modifiées rétroactivement.
- **Solution :** Création d'une table `LigneFacture` qui stocke le prix et le libellé de la prestation **au moment précis de la vente**. On ne se contente pas de pointer vers la table des tarifs.

### 3. Workflow de Validation Médicale (Le Double Verrou)

- **Problématique :** Le médecin ne doit pas interpréter des résultats non vérifiés.
- **Solution :** Ajout d'un cycle de vie pour les examens :
  1. PRESCRIT (Médecin)
  2. PAYÉ (Caissier)
  3. RÉALISÉ (Laborantin)
  4. VALIDÉ (Responsable Labo) -> *C'est à ce stade seulement que le médecin accède aux résultats.*

### 4. Traçabilité et Audit Trail (Zéro Suppression)

- **Problématique :** Risque de fraude si une facture est simplement supprimée.
- **Solution :** Utilisation du Soft Delete et de l'annulation par **Avoir**. Toute erreur de caisse doit faire l'objet d'une facture d'annulation pour que l'historique reste auditable par l'administrateur.

### 5. Choix Techniques Backend

- **Framework :** Django.

- **Sécurité** : Utilisation des Atomic Transactions de Django pour garantir que si la création d'une facture échoue, aucun paiement n'est enregistré (tout-ou-rien).
- **Automatisation** : Utilisation des Django Signals pour mettre à jour automatiquement le statut des prestations dès qu'une facture est soldée.

**Schéma de données mis à jour (Résumé) :**

- **Table Prescription** : Ajout des champs `is_paid` (bool) et `status` (choice field).
- **Table Facture** : Ajout d'une référence unique (ex: FAC-2025-XXXX) et lien vers le Caissier.
- **Table LigneFacture** : Table de liaison avec prix figé.

**Souhaitez-vous que je développe maintenant**